

ECUE 425 – Mécanismes internes des systèmes de santé

Socket IO

- Bibliothèque JavaScript pour communication bidirectionnelle :
 - En temps réel
 - Basée sur les événements.
- Multiplateforme
 - Chrome, Firefox, Safari, Edge
- Il se compose de deux parties:
 - une bibliothèque côté client qui s'exécute dans le navigateur et
 - une bibliothèque côté serveur pour node.js.
 - Les deux composants ont une API identique.

- Applications en temps réel :
 - Messagers instantanés - Applications de chat comme Whatsapp, Facebook Messenger, etc. Vous n'avez pas besoin d'actualiser votre application / site Web pour recevoir de nouveaux messages.
 - Notifications push - Lorsque quelqu'un vous identifie sur une photo sur Facebook, vous recevez une notification instantanément.
 - Applications de collaboration - Des applications telles que google docs, qui permettent à plusieurs personnes de mettre à jour les mêmes documents simultanément et d'appliquer des modifications aux instances de toutes les personnes.
 - Jeux en ligne - Des jeux comme Counter Strike, Call of Duty, etc., sont également quelques exemples d'applications en temps réel.

- Avant utilisation, l'installation :
 - Node.JS : environnement pour exécuter du code JavaScript
 - <https://nodejs.org/en/download/>
 - npm (node package manager) : gestionnaire de paquets pour NodeJS et Javascript
 - Installé avec Node.JS
 - <https://www.npmjs.com/get-npm>

```
socketIO — -bash — 80x24
(base) thiago:socketIO$ node --version
v14.16.0
(base) thiago:socketIO$ npm --version
7.5.6
(base) thiago:socketIO$
```

Hello World

6

- Hello World
 - Nous avons d'un dossier dédié pour notre projet (propre)
 - Nous indiquerons les dépendances nécessaires :
 - Express Web Server : serveur web léger en JavaScript (comme Flask en Python)
 - npm install express
 - Socket IO
 - npm install socket.io
 - Pour plus de détails sur le dépendences et JSON :
 - <https://flaviocopes.com/package-json/>

Hello World

- « app.js »
 - node app.js
- « express » initialise l'application pour être un gestionnaire de fonctions que vous pouvez fournir à un serveur HTTP (comme illustré à la ligne 2).
- Nous définissons un gestionnaire d'itinéraire / qui est appelé lorsque nous accédons à la page d'accueil de notre site Web.
- Nous faisons écouter le serveur http sur le port 3000.
 - npm init
 - Pour définir les attributs du paquet

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index.html');
7 });
8
9 http.listen(3000, function() {
10   console.log('listening on *:3000');
11 });
```

```
[(base) MacBook-Pro-de-Thiago-2:socketIO thiago$ node app.js
listening on *:3000
```

Hello World

8

- « index.html »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <body>Hello world</body>
7 </html>
```

← → ↻ ⓘ localhost:3000

Hello world

Hello World

- On intègre le module « Socket IO »
 - On crée une instance de SocketIO « io » en fournissant le serveur « http » comme argument
 - La fonction « on() » écoute les événements : c'est une fonction de callback de SocketIO
 - Notez que la fonction de callback io.on() accepte deux arguments :
 - « connection » indiquant le type d'action
 - « function(socket) » une fonction qui affiche sur la console
- A chaque nouvel utilisateur, le message « a user connected » sera affichée.
- Il faut éditer le html également

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index.html');
7 });
8
9 //Whenever someone connects this gets executed
10 io.on('connection', function(socket) {
11   console.log('A user connected');
12
13 });
14
15 http.listen(3000, function() {
16   console.log('listening on *:3000');
17 });
```

Hello World

10

- index.html
 - On charge le socket.io-client, qui définit une classe io global (et le point de terminaison GET /socket.io/socket.io.js), pour se connecter.
 - Ce fichier est également installé dans le dossier node_modules. Vous pouvez remplacer le chemin de la ligne 6 par :
 - node_modules/socket.io-client/dist/socket.io.js
 - Chaque page ouverte sur localhost forcera le message « a user connected »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7
8   <script>
9     var socket = io();
10  </script>
11  <body>Hello world</body>
12 </html>
```

```
((base) MacBook-Pro-de-Thiago-2:socketIO thiago$ node app.js
listening on *:3000
A user connected
```

Hello World

- « app.js »
 - Notez que la fonction définie en « function(socket) » possède un paramètre « socket »
 - Ce paramètre traite les événements des utilisateurs connectés et possède ses fonctions de callback pour gérer leurs propres événements
 - Chaque fois qu'un utilisateur se déconnecte, le message « a user disconnected » sera affiché

```

1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index.html');
7 });
8
9 //Whenever someone connects this gets executed
10 io.on('connection', function(socket) {
11   console.log('A user connected');
12   //Whenever someone disconnects this piece of code executed
13
14   socket.on('disconnect', function () {
15     console.log('A user disconnected');
16   });
17
18 });
19
20 http.listen(3000, function() {
21   console.log('listening on *:3000');
22 });

```

```

(base) MacBook-Pro-de-Thiago-2:socketIO thiago$ node app.js
listening on *:3000
A user connected
A user disconnected
A user connected
A user connected

```

- Sockets sont basés sur des événements
- Coté serveur :
 - Connection
 - Message
 - Disconnect
 - Reconnect
 - Ping
 - Join and
 - Leave
- Sockets sont basés sur des événements
- Coté client :
 - Connection
 - Connect_error
 - Connect_timeout
 - Reconnect, etc

- Comment échanger des messages entre client et serveur ?
- Deux possibilités :
 - `socket.send()` → déclenche un événement « message » (interne) qui peut être capturé de l'autre côté
 - `socket.emit(« unNomQuelconque »)` → déclenche un événement « unNomQuelconque » qui peut être capturé de l'autre côté

SocketIO - Send

- socket.send()
- app1Send.js

Après 4000ms on envoie
un « message »

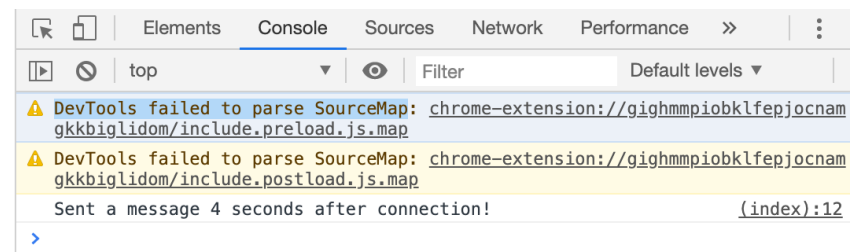
```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index1Send.html');
7 });
8
9 //Whenever someone connects this gets executed
10 io.on('connection', function(socket) {
11   console.log('A user connected');
12
13   //Send a message after a timeout of 4seconds
14   setTimeout(function() {
15     socket.send('Sent a message 4 seconds after connection!');
16   }, 4000);
17
18   //Whenever someone disconnects this piece of code executed
19   socket.on('disconnect', function () {
20     console.log('A user disconnected');
21   });
22
23 });
24
25 http.listen(3000, function() {
26   console.log('listening on *:3000');
27 });
```

SocketIO - Send

- socket.send()
- index1Send.html

On capture les événements du type « message »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7
8   <script>
9     var socket = io();
10    socket.on('message', function(data){
11      console.log(data);
12    });
13  </script>
14
15  <body>Hello world</body>
16 </html>
```



SocketIO - Emit

- socket.emit()
- app1Emit.js

Après 4000ms on envoie
un « testEvent »

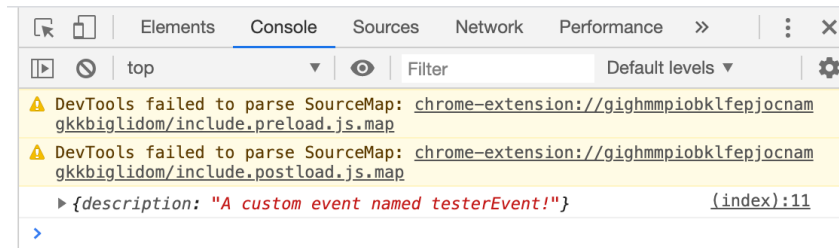
```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index1Emit.html');
7 });
8
9 //Whenever someone connects this gets executed
10 io.on('connection', function(socket) {
11   console.log('A user connected');
12
13   //Send a message after a timeout of 4 seconds
14   setTimeout(function() {
15     //Sending an object when emitting an event
16     socket.emit('testEvent', {
17       description: 'A custom event named testerEvent!'});
18     //socket.send('Sent a message 4 seconds after connection!');
19   }, 4000);
20
21   //Whenever someone disconnects this piece of code executed
22   socket.on('disconnect', function () {
23     console.log('A user disconnected');
24   });
25
26 });
27
28 http.listen(3000, function() {
29   console.log('listening on *:3000');
30 });
```


SocketIO - Emit

- socket.emit()
- index1Emit.html

On capture les événements du type « testEvent »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7
8   <script>
9     var socket = io();
10    socket.on('testEvent', function(data){
11      console.log(data);
12    });
13  </script>
14
15  <body>Hello world</body>
16 </html>
```



SocketIO - Emit

- socket.emit()
 - app2Emit.js
- Maintenant le client envoie message

Écoute l'événement « ClientEvent »

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/index2Emit.html');
7 });
8
9 //Whenever someone connects this gets executed
10 io.on('connection', function(socket) {
11   console.log('A user connected');
12
13   socket.on('clientEvent', function(data) {
14     console.log(data);
15   });
16
17   //Whenever someone disconnects this piece of code executed
18   socket.on('disconnect', function () {
19     console.log('A user disconnected');
20   });
21 });
22
23
24 http.listen(3000, function() {
25   console.log('listening on *:3000');
26 });
```

SocketIO - Emit

19

- socket.emit()
 - index2Emit.html

Emet l'événement « clientEvent »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7
8   <script>
9     var socket = io();
10    socket.emit('clientEvent', 'Sent an event from the client!');
11  </script>
12
13  <body>Hello world</body>
14 </html>
```

```
[(base) MacBook-Pro-de-Thiago-2:socketIO thiago$ node app2Emit.js
listening on *:3000
A user connected
A user disconnected
A user connected
Sent an event from the client!
_
```

SocketIO - Broadcast

20

- Faire de la diffusion 1 (broadcast)
 - socket.emit
 - Tous reçoivent l'info, y compris l'émetteur

On augmente le nombre de clients connectés

On diminue le nombre de clients connectés

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/indexBroadcast.html');
7 });
8
9 var clients = 0;
10 io.on('connection', function(socket) {
11   clients++;
12   io.sockets.emit('broadcast',{
13     description: clients + ' clients connected!'});
14   socket.on('disconnect', function () {
15     clients--;
16     io.sockets.emit('broadcast',{
17       description: clients + ' clients connected!'});
18   });
19 });
20
21 http.listen(3000, function() {
22   console.log('listening on localhost:3000');
23 });
```

21/03/2024

SocketIO - Broadcast

- Faire de la diffusion 1 (broadcast)
 - socket.emit
 - Tous reçoivent l'info, y compris l'émetteur

On écrit sur le HTML
(DOM JavaScript)

```
2 <html>
3   <head>
4     <title>Hello world</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7   <script>
8     var socket = io();
9     socket.on('broadcast',function(data) {
10       document.body.innerHTML = '';
11       document.write(data.description);
12     });
13   </script>
14   <body>Hello world</body>
15 </html>
```

Ce message sera affiché
dans les deux fenêtres

← → ↻ ⓘ localhost:3000

2 clients connected!

SocketIO - Broadcast

22

- Faire de la diffusion 2 (broadcast)
 - socket.broadcast.emit
 - Tous reçoivent l'info, sauf l'émetteur

Tous reçoivent « Welcome »

Les autres reçoivent le nombre d'utilisateurs

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/indexBroadcast.html');
7 });
8
9 var clients = 0;
10 io.on('connection', function(socket) {
11   clients++;
12   socket.emit('newclientconnect',{ description: 'Hey, welcome!'});
13   socket.broadcast.emit('newclientconnect',{
14     description: clients + ' clients connected!'});
15
16   socket.on('disconnect', function () {
17     clients--;
18     io.sockets.emit('broadcast',{
19       description: clients + ' clients connected!'});
20   });
21 });
22
23 http.listen(3000, function() {
24   console.log('listening on localhost:3000');
25 });
```

21/03/2024

SocketIO - Broadcast

- Faire de la diffusion 2 (broadcast)
 - socket.broadcast.emit
 - Tous reçoivent l'info, sauf l'émetteur

« newclientconnect »

Nouveaux arrivants

Anciens membres

```

1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Hello world</title>
5    </head>
6    <script src = "/socket.io/socket.io.js"></script>
7    <script>
8      var socket = io();
9      socket.on('newclientconnect',function(data) {
10         document.body.innerHTML = '';
11         document.write(data.description);
12      });
13    </script>
14    <body>Hello world</body>
15  </html>

```

← → × ⓘ localhost:3000

Hey, welcome!

← → ↻ ⓘ localhost:3000

2 clients connected!

- Namespace

- Un « nom » définissant le groupe de connexion
- Indique le point de terminaison auquel le client peut se connecter
- Par défaut « / »

Nouveau namespace
« my-namespace »

```
1 var app = require('express')();
2 var http = require('http').Server(app);
3 var io = require('socket.io')(http);
4
5 app.get('/', function(req, res) {
6   res.sendFile(__dirname + '/indexNamespace.html');
7 });
8
9 var nsp = io.of('/my-namespace');
10 nsp.on('connection', function(socket) {
11   console.log('someone connected');
12   nsp.emit('hi', 'Hello everyone!');
13 });
14
15 http.listen(3000, function() {
16   console.log('listening on localhost:3000');
17 });
```


- Namespace
 - Un « nom » définissant le groupe de connexion
 - Indique le point de terminaison auquel le client peut se connecter
 - Par défaut « / »

Client se connecte sur
« my-namespace »

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Namespaces</title>
5   </head>
6   <script src = "/socket.io/socket.io.js"></script>
7   <script>
8     var socket = io('/my-namespace');
9     socket.on('hi',function(data) {
10       document.body.innerHTML = '';
11       document.write(data);
12     });
13   </script>
14   <body>Hello world</body>
15 </html>
```

SocketIO – Salles (Rooms)

- Rooms

- Des canaux arbitraires pour établir de groupes dans un namespace
- Méthode « join » permet d'accéder à une salle

Chaque salle aura au maximum deux membres, avant qu'une nouvelle salle soit créée

« join » pour connecter à la salle

On fait un broadcast aux membres de la salle

```
JS appRooms.js > ...
1  var app = require('express')();
2  var http = require('http').Server(app);
3  var io = require('socket.io')(http);
4
5  app.get('/', function(req, res) {
6    res.sendFile(__dirname + '/indexRooms.html');
7  });
8
9  var roomno = 1;
10 io.on('connection', function(socket) {
11    // attention : si vous rencontrez le problème de "undefined"
12    // veuillez remplacer le IF par celui ci
13    // (cela marche pour version de socket.io > 3):
14    /* if (io.sockets.adapter.rooms.get("room-" + roomno) &&
15        io.sockets.adapter.rooms.get("room-" + roomno).size > 1 {
16        roomno ++;
17    }
18    */
19    //Increase roomno 2 clients are present in a room.
20    if(io.nsp['/'].adapter.rooms["room-"+roomno] &&
21        io.nsp['/'].adapter.rooms["room-"+roomno].length > 1) {
22        roomno++;
23    }
24    socket.join("room-"+roomno);
25
26    //Send this event to everyone in the room.
27    io.sockets.in("room-"+roomno).emit('connectToRoom',
28        "You are in room no. "+roomno);
29  })
30
31  http.listen(3000, function() {
32    console.log('listening on localhost:3000');
33  });
```

SocketIO – Salles (Rooms)

- Rooms

- Des canaux arbitraires pour établir de groupes dans un namespace
- Méthode « join » permet d'accéder à une salle

Du côté HTML, rien ne change

```

1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Rooms</title>
5    </head>
6    <script src = "/socket.io/socket.io.js"></script>
7    <script>
8      var socket = io();
9      socket.on('connectToRoom',function(data) {
10         document.body.innerHTML = '';
11         document.write(data);
12       });
13    </script>
14    <body>Rooms</body>
15  </html>

```

← → ↻ ⓘ localhost:3000

You are in room no. 1

← → ↻ ⓘ localhost:3000

You are in room no. 1

← → ↻ ⓘ localhost:3000

You are in room no. 2

Obtenir IP client

28

```
1  var app = require('express')();
2  var http = require('http').Server(app);
3  var io = require('socket.io')(http);
4
5  const host = 'localhost';
6  const port = 3000;
7
8  app.get('/', function(req, res) {
9    res.sendFile(__dirname + '/index.html');
10 });
11
12 //Whenever someone connects this gets executed
13 io.on('connection', function(socket) {
14   console.log('A user connected');
15
```

```
15
16   var socketId = socket.id;
17   var clientIp = socket.request.connection.remoteAddress;
18   var clientIpSplit = clientIp.split(":")[3]; //enlever le format ipv6
19   console.log(socketId);
20   console.log(clientIp); //format ipv6+ipv4
21   console.log(clientIpSplit); //enlever le format ipv6
22
23   //Whenever someone disconnects this piece of code executed
24   socket.on('disconnect', function () {
25     console.log('A user disconnected');
26   });
27
28 });
29
30 http.listen(3000, function() {
31   console.log('listening on *:3000');
32 });
```

```
[(base) thiago:socketIO$ node ip-handshake-example.js
listening on *:3000
A user connected
YTwwqMWHR2Vndv2AAAAA ← Ligne 19
::ffff:192.168.1.44 ← Ligne 20
192.168.1.44 ← Ligne 21
```

- 1) Améliorez l'exemple précédent sur les « Salles » (« Rooms ») pour afficher le nombre d'utilisateurs dans une salle
 - Attention, si quelqu'un se connecte ou déconnecte de la salle, le nouveau nombre de gens sera affiché
 - Deux utilisateurs par salle
 - Si tous les utilisateurs d'une salle l'abandonne, on repeupler cette salle avant de lancer une nouvelle
- 2) Écrivez une application de messagerie instantanée (« tchat ») en utilisant Socket IO

SocketIO – Références

- <https://socket.io/get-started/chat/>
- https://www.tutorialspoint.com/socket.io/socket.io_rooms.htm
- <https://flask-socketio.readthedocs.io/en/latest/>